

Universidad del Valle de Guatemala

[Facultad / Escuela — completar según la carátula oficial del curso]

Screensaver Paralelo: Dino Autojutable

Paralelización con OpenMP de un ciclo de simulación de partículas de
clima

Proyecto #1 — Computación Paralela y Distribuida

Docente: Marlon Fuentes

Integrantes:

Mario Rocha

[Nombre Integrante 2] — completar

[Nombre Integrante 3] — completar

Semestre 2, 2026

Guatemala, 9 de septiembre de 2026

Índice

1. Introducción	2
2. Antecedentes	2
2.1. OpenMP	2
2.2. El método PCAM	3
2.3. Speedup, eficiencia y la ley de Amdahl	3
3. Diseño e implementación	3
3.1. Descripción del screensaver	3
3.2. Arquitectura compartida	4
3.3. Aplicación del método PCAM	4
3.4. Implementación: secuencial vs. paralela	5
3.5. Metodología de pruebas	6
3.6. Resultados	6
4. Conclusiones	7
A. Apéndice: material suplementario	9
A.1. Anexo 1 — Diagrama de flujo	9
A.2. Anexo 2 — Catálogo de funciones	10
A.3. Anexo 3 — Bitácora de pruebas	11
Referencias bibliográficas	13

1. Introducción

Este informe documenta el desarrollo del Proyecto #1 del curso de Computación Paralela y Distribuida (UVG, semestre 2, 2026): un *screensaver* autoejecutable estilo “Dino de Chrome”, implementado en C con la biblioteca gráfica SDL2, cuyo componente central — la simulación física de partículas de clima — se paraleliza con OpenMP.

El objetivo del proyecto no es únicamente producir una animación visual, sino aplicar de forma práctica los conceptos de paralelización con memoria compartida vistos en el curso: identificar qué parte de un programa secuencial es paralelizable, aplicar el método PCAM (Partición, Comunicación, Aglomeración, Mapeo) para diseñar su versión paralela, implementarla con OpenMP, y medir su desempeño real en términos de *speedup* y eficiencia frente a la versión secuencial.

El documento se organiza así: la sección de antecedentes resume los conceptos teóricos usados (OpenMP, PCAM, speedup y eficiencia, ley de Amdahl); el cuerpo describe el diseño del screensaver, la arquitectura del código compartido entre las versiones secuencial y paralela, la estrategia de paralelización aplicada y los resultados medidos; y las conclusiones resumen los hallazgos y su relación con la teoría. Los anexos — diagrama de flujo, catálogo de funciones y bitácora de pruebas — se incluyen como apéndice con el material de soporte.

2. Antecedentes

2.1. OpenMP

OpenMP es una interfaz de programación de aplicaciones (API) para programación paralela con memoria compartida en C, C++ y Fortran, mantenida por el OpenMP Architecture Review Board. Se basa en un modelo de ejecución *fork-join*: un hilo maestro ejecuta el programa de forma secuencial hasta encontrar una directiva de compilador (un `#pragma omp`) que abre una región paralela, momento en el que se crea un equipo de hilos que ejecutan el trabajo repartido entre ellos hasta un punto de sincronización implícito al cerrar la región, tras el cual el hilo maestro continúa solo.¹ Esta característica es justamente la que permite paralelizar un `for` secuencial con una sola línea (`#pragma omp parallel for`) sin reescribir la lógica interna del ciclo, siempre que las iteraciones sean independientes entre sí.

Para acumular un valor entre iteraciones sin condiciones de carrera — como un contador que varios hilos incrementan —, OpenMP ofrece la cláusula `reduction`, que le da a cada hilo una copia privada de la variable, la inicializa según el operador indicado, y combina todas las copias en la variable original al finalizar la región paralela. Esto evita usar una

¹OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. 2021. Disponible en <https://www.openmp.org>.

sección crítica (`#pragma omp critical`) que serializaría esa parte del ciclo y anularía buena parte del beneficio de paralelizar.

2.2. El método PCAM

PCAM es una metodología de diseño de programas paralelos propuesta por Ian Foster, organizada en cuatro etapas: **Partición** (descomponer el problema en tareas lo más pequeñas e independientes posible), **Comunicación** (identificar qué datos deben intercambiarse entre tareas), **Aglomeración** (agrupar tareas pequeñas en unidades más grandes para reducir el overhead de comunicación/sincronización) y **Mapeo** (asignar las tareas agrupadas a los procesadores o hilos disponibles).² La sección 3.3 de este informe aplica explícitamente estas cuatro etapas al diseño de `particles_update()`.

2.3. Speedup, eficiencia y la ley de Amdahl

El *speedup* de una versión paralela con p procesadores/hilos se define como $S_p = T_1/T_p$, donde T_1 es el tiempo de la versión secuencial y T_p el de la versión paralela con p hilos; la *eficiencia* es $E_p = S_p/p$, y mide qué tan bien se aprovecha cada hilo adicional (1.0 sería aprovechamiento perfecto).³ Gene Amdahl formalizó en 1967 la idea de que el speedup máximo alcanzable está acotado por la fracción del programa que *no* se puede paralelizar: si esa fracción secuencial es pequeña pero fija, agregar más procesadores tiene rendimientos cada vez menores y en algún punto el overhead de coordinar más hilos puede superar la ganancia.⁴ Este proyecto observa ese efecto directamente: como se detalla en la sección 3.6, el *speedup* deja de crecer (e incluso empeora) al pasar de 4 a 8 hilos, precisamente porque el trabajo por hilo se vuelve pequeño frente al overhead fijo de sincronización por frame.

3. Diseño e implementación

3.1. Descripción del screensaver

El screensaver dibuja un dinosaurio que salta obstáculos de forma autónoma (regla determinista de distancia, sin inteligencia artificial), sobre un ciclo de día/noche y estaciones (primavera, verano, otoño, invierno) que cambian la paleta de color de la escena mediante interpolación e integran una componente trigonométrica (`sinf()`) sobre el tiempo transcurrido). El parámetro requerido por la rúbrica del curso es N : la cantidad de partículas

²Foster, I. *Designing and Building Parallel Programs*. Addison-Wesley, 1995. Disponible en <https://www.mcs.anl.gov/~itf/dbpp/>.

³Grama, A., Gupta, A., Karypis, G., & Kumar, V. *Introduction to Parallel Computing* (2.^a ed.). Addison-Wesley, 2003.

⁴Amdahl, G. M. “Validity of the single processor approach to achieving large scale computing capabilities”. *AFIPS Conference Proceedings*, vol. 30, 1967, pp. 483–485.

de clima (lluvia, nieve u hojas según la estación) que caen con física real — gravedad constante más viento lateral sinusoidal — y rebotan en los bordes laterales del lienzo. El lienzo es configurable por línea de comandos con un mínimo de 640×480 px, cumpliendo el requisito mínimo del enunciado. La actualización de esas N partículas por frame es, deliberadamente, la única parte del programa que se mide y paraleliza — el resto (lógica del dino, obstáculos, render) se mantiene simple e intencionalmente secuencial para concentrar el esfuerzo de paralelización donde realmente hay trabajo independiente y repetitivo.

3.2. Arquitectura compartida

Las versiones secuencial (`screensaver_seq`) y paralela (`screensaver_par`) comparten prácticamente todo el código, en `src/common/`: parseo de argumentos, física de una partícula individual (`particle_step`), lógica del dino y los obstáculos, cálculo de la paleta día/noche/estación, y el render con SDL2. La única diferencia entre binarios es la función `particles_update()`, inyectada como puntero a función (`ParticleUpdateFn`) al loop principal compartido `run_app()` (`src/common/app.c`):

Listing 1: Firma compartida entre la versión seq y par (`include/app.h`)

```
1 typedef int (*ParticleUpdateFn)(Particle *particles, int n,
2                               const SimState *state, float dt);
3 int run_app(const Config *cfg, ParticleUpdateFn update_fn);
```

`run_app()` soporta dos modos: interactivo (ventana SDL, loop hasta cerrar o alcanzar `-frames`) y `-benchmark` (headless, sin SDL, corre exactamente `-frames` pasos y mide solo el tiempo del loop de partículas). El modo benchmark es el que se usó para todas las mediciones de la sección 3.6 y del Anexo 3, porque aísla la sección paralelizada del resto del programa (que nunca se paraleliza y no debe contaminar la medición).

3.3. Aplicación del método PCAM

Partición

Cada partícula de clima es una tarea independiente: su próximo estado (`x`, `y`, `vx`, `vy`) depende únicamente de su propio estado actual y de `SimState` (de solo lectura), nunca del estado de otra partícula — no hay colisiones entre partículas por diseño. Esto da una partición natural de grano fino: una tarea por partícula, hasta N tareas totalmente independientes por frame.

Comunicación

Es mínima: cada tarea solo lee `SimState` (compartido, de solo lectura, sin necesidad de proteger) y escribe únicamente su propio elemento del arreglo `particles[i]`. El único dato realmente compartido *en escritura* es el contador de partículas recicladas en el frame (`respawned`).

Aglomeración

Agrupar cada partícula en su propia tarea de sistema operativo sería absurdo (miles de hilos por frame). En vez de eso, OpenMP con `schedule(static)` agrupa las N partículas en bloques contiguos, uno por hilo disponible — la aglomeración la decide el *runtime* de OpenMP, no el código de la aplicación.

Mapeo

`#pragma omp parallel for` mapea cada bloque aglomerado a un hilo del equipo creado por OpenMP; el número de hilos lo decide el sistema operativo por default, o el usuario vía `-threads N` (que llama `omp_set_num_threads()` antes de correr el loop).

3.4. Implementación: secuencial vs. paralela

Listing 2: Version secuencial (src/seq/particle_update.c)

```

1 int particles_update(Particle *particles, int n,
2                     const SimState *state, float dt) {
3     int respawned = 0;
4     for (int i = 0; i < n; i++) {
5         if (particle_step(&particles[i], state, dt)) {
6             respawned++;
7         }
8     }
9     return respawned;
10 }
```

Listing 3: Version paralela con OpenMP (src/par/particle_update.c)

```

1 int particles_update(Particle *particles, int n,
2                     const SimState *state, float dt) {
3     int respawned = 0;
4
5     #pragma omp parallel for schedule(static) reduction(+:respawned)
6     for (int i = 0; i < n; i++) {
7         if (particle_step(&particles[i], state, dt)) {
8             respawned++;
9         }
10    }
11    return respawned;
12 }
```

La única diferencia de código es la directiva `#pragma omp parallel for schedule(static) reduction(+:respawned)`: no hace falta ningún lock ni sección crítica explícita porque (a) las partículas no comparten estado entre sí, y (b) cada partícula lleva su propio generador de números aleatorios (`rand_r` con `rng_state` propio, sembrado con un multiplicador tipo hash de Knuth por índice) — así que tampoco hay una semilla global que sincronizar

entre hilos, un problema común al paralelizar código que depende de `rand()` clásico. El mecanismo de sincronía de esta versión es exclusivamente la cláusula `reduction`, que combina el acumulador local de `respawned` de cada hilo al cerrar la región paralela.

Programación defensiva: `parse_args()` valida tipos (enteros bien formados vía `strtol`), rangos (`n>0`, lienzo mínimo 640×480 , `fps>0`, `frames>=0`, `threads>=0`) y una regla cruzada (`-benchmark` exige `-frames>0` para que la medición sea reproducible), todo antes de reservar memoria o inicializar SDL; `particles_create()` revisa el retorno de `malloc()`; y todas las rutas de inicialización de SDL (ventana, renderer) revisan su valor de retorno y liberan lo ya reservado antes de salir con código de error.

3.5. Metodología de pruebas

Las mediciones se hicieron con el modo `-benchmark` de ambos binarios en la misma máquina (procesador Apple Silicon, 8 núcleos lógicos), corriendo 300 frames por medición, 10 repeticiones por cada combinación de $N \in \{500, 2000, 8000, 20000, 50000\}$ y, en la versión paralela, hilos $\in \{1, 2, 4, 8\}$ — 250 mediciones en total. El detalle completo, la captura de la corrida y el análisis están en el Anexo 3 (sección A.3).

3.6. Resultados

Cuadro 1: Speedup y eficiencia promedio por combinación (N, hilos)

N	Hilos	T. secuencial (ms)	T. paralelo (ms)	Speedup (eficiencia)
8000	4	14.531	6.077	2.39 (0.60)
20000	4	36.819	12.367	2.98 (0.74)
50000	4	93.322	28.305	3.30 (0.82)
50000	8	93.322	37.414	2.49 (0.31)
500	8	0.974	9.921	0.10 (0.01)

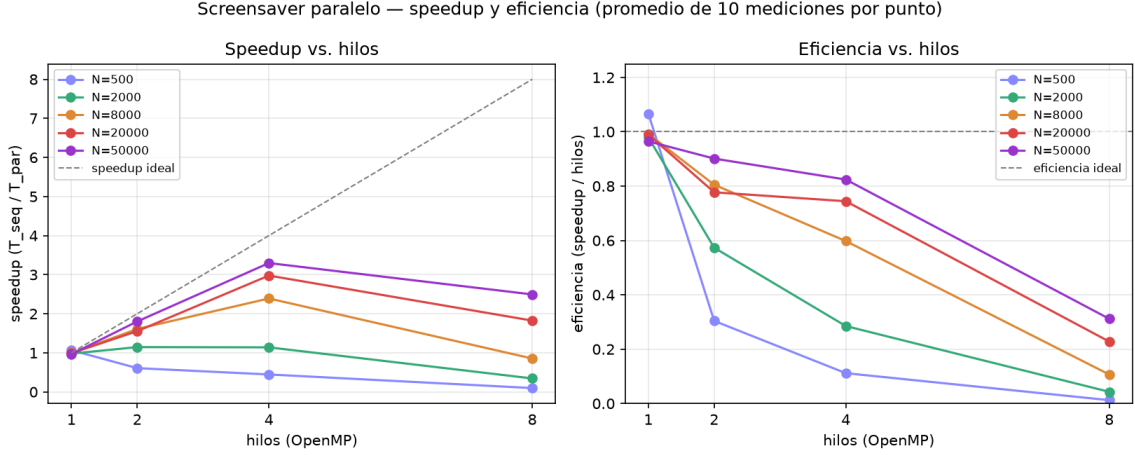


Figura 1: Speedup y eficiencia medidos vs. número de hilos, por tamaño de N

El mejor resultado medido es un **speedup de 3.30x con 4 hilos y N=50000** (eficiencia 0.82). Con N pequeño (500) el paralelo es más lento que el secuencial: el overhead de OpenMP de coordinar hilos en cada uno de los 300 `fork-join` por frame supera el trabajo real cuando hay pocas partículas. Y en todos los tamaños de N probados, **8 hilos rinde peor que 4** — consistente con la sección de antecedentes sobre la ley de Amdahl: el overhead fijo por frame pesa proporcionalmente más cuantos más hilos compiten por coordinarse, y en un chip con núcleos de rendimiento y eficiencia heterogéneos, forzar 8 hilos reparte trabajo también sobre núcleos más lentos. El análisis completo, con la tabla de las 20 combinaciones medidas, está en el Anexo 3.

Se verificó además que paralelizar no altera el resultado de la simulación: con semilla fija (`-seed 12345`, `N=8000`), la versión secuencial y la paralela con 1, 4 y 8 hilos reciclan exactamente **26144** partículas en 300 frames — confirmando que `reduction(+:respawned)` combina los contadores sin perder ni duplicar eventos, y que el orden de procesamiento entre hilos no afecta el resultado (ver Anexo 3).

4. Conclusiones

1. Paralelizar con OpenMP el loop de actualización de partículas fue directo porque el problema tiene una partición natural de grano fino (partículas independientes) con comunicación mínima (solo un contador compartido), lo que permitió resolverlo con una sola directiva `#pragma omp parallel for` y una `reduction`, sin necesidad de locks explícitos.
2. El speedup obtenido depende fuertemente de N: solo se observa una ganancia real (hasta 3.30x) cuando hay suficiente trabajo por frame para amortizar el overhead fijo de crear/sincronizar hilos; con pocas partículas, paralelizar activamente perjudica el desempeño.

3. Más hilos no es necesariamente mejor: 4 hilos superó consistentemente a 8 en esta máquina, lo que confirma en la práctica la intuición de la ley de Amdahl y sugiere que la elección del número de hilos debe basarse en mediciones, no asumirse igual al máximo de núcleos disponibles.
4. Se verificó que la paralelización no introdujo condiciones de carrera: con semilla fija, el resultado de la simulación (partículas recicladas) es idéntico entre la versión secuencial y la paralela, en cualquier número de hilos probado.

Recomendaciones

- Para trabajo futuro, valdría la pena medir con `schedule(dynamic)` en vez de `schedule(static)` para ver si compensa mejor el desbalance entre núcleos de rendimiento y eficiencia del chip usado.
- Repetir las mediciones en una máquina con núcleos homogéneos (p. ej. un servidor Linux) ayudaría a aislar si la caída de eficiencia a 8 hilos es overhead de OpenMP puro o un efecto específico de la heterogeneidad de núcleos de este equipo.

A. Apéndice: material suplementario

A.1. Anexo 1 — Diagrama de flujo

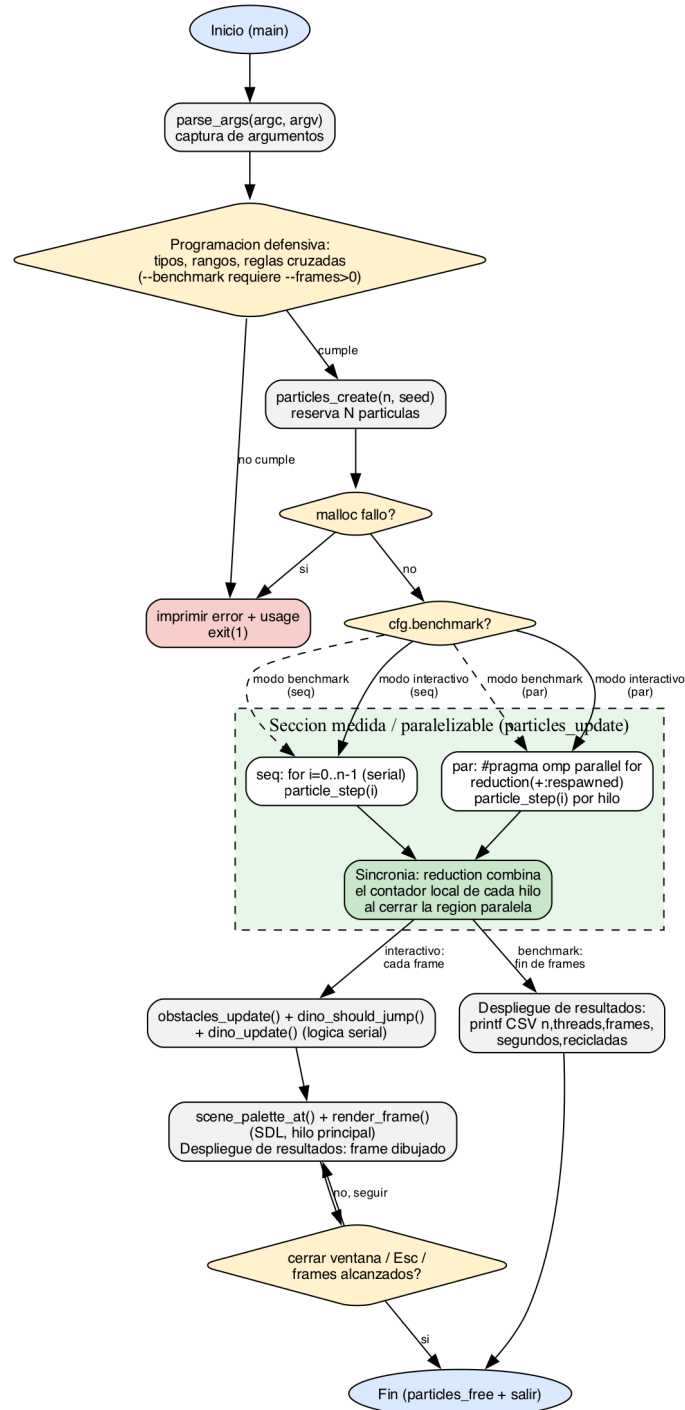


Figura 2: Diagrama de flujo del programa (aplica a `screensaver_seq` y `screensaver_par`)

La versión detallada de este diagrama (con todas las ramas de validación defensiva) está en `docs/anexo1_diagrama_flujo.md` del repositorio.

A.2. Anexo 2 — Catálogo de funciones

El catálogo completo de funciones del proyecto (entradas, salidas y descripción de cada una, incluyendo las funciones `static`) está en `docs/anexo2_catalogo_funciones.md` del repositorio, por su extensión no se transcribe aquí completo. Resumen de las funciones centrales para la paralelización:

Función	Entradas / Salidas	Descripción
<code>particle_step</code>	Entra: <code>Particle*</code> , <code>SimState*</code> , <code>dt</code> . Sale: <code>bool</code> (¿se recicló?).	Física de una partícula: viento sinusoidal + gravedad, rebote en bordes, reciclaje al salir por abajo. Unidad de trabajo independiente paralelizada.
<code>particles_update</code> (seq)	Entra: arreglo de <code>N</code> partículas, <code>SimState*</code> , <code>dt</code> . Sale: <code>int</code> partículas recicladas.	<code>for</code> secuencial que llama <code>particle_step</code> por cada partícula.
<code>particles_update</code> (par)	Misma firma que la versión seq.	Igual, pero con <code>#pragma omp parallel for schedule(static) reduction(+:respawned)</code> .

A.3. Anexo 3 — Bitácora de pruebas

Captura de la corrida de benchmarks

```
screensaver - zsh - 90x30

$ ./scripts/run_benchmarks.sh
seq n=500: 10 mediciones listas
par n=500 threads=1: 10 mediciones listas
par n=500 threads=2: 10 mediciones listas
par n=500 threads=4: 10 mediciones listas
par n=500 threads=8: 10 mediciones listas
seq n=2000: 10 mediciones listas
par n=2000 threads=1: 10 mediciones listas
par n=2000 threads=2: 10 mediciones listas
par n=2000 threads=4: 10 mediciones listas
par n=2000 threads=8: 10 mediciones listas
seq n=8000: 10 mediciones listas
par n=8000 threads=1: 10 mediciones listas
par n=8000 threads=2: 10 mediciones listas
par n=8000 threads=4: 10 mediciones listas
par n=8000 threads=8: 10 mediciones listas
seq n=20000: 10 mediciones listas
par n=20000 threads=1: 10 mediciones listas
par n=20000 threads=2: 10 mediciones listas
par n=20000 threads=4: 10 mediciones listas
par n=20000 threads=8: 10 mediciones listas
seq n=50000: 10 mediciones listas
par n=50000 threads=1: 10 mediciones listas
par n=50000 threads=2: 10 mediciones listas
par n=50000 threads=4: 10 mediciones listas
par n=50000 threads=8: 10 mediciones listas

Listo: 250 corridas guardadas en results/benchmark_results.csv

$ head -11 results/benchmark_results.csv
version,n,threads,frames,seconds,respawned,rep
seq,500,0,300,0.000888,1642,1
seq,500,0,300,0.001212,1642,2
seq,500,0,300,0.001131,1642,3
seq,500,0,300,0.001030,1642,4
seq,500,0,300,0.000986,1642,5
seq,500,0,300,0.000954,1642,6
seq,500,0,300,0.000912,1642,7
seq,500,0,300,0.000875,1642,8
seq,500,0,300,0.000917,1642,9
seq,500,0,300,0.000834,1642,10
```

Figura 3: Corrida real de `scripts/run_benchmarks.sh` y primeras filas del CSV crudo generado (250 mediciones, 10 repeticiones por combinación de N y número de hilos)

Tabla completa de resultados (promedio de 10 mediciones por fila)

N	Hilos	T. secuencial (ms)	T. paralelo (ms)	Speedup	Eficiencia
500	1	0.974	0.913	1.07	1.07
500	2	0.974	1.602	0.61	0.30
500	4	0.974	2.182	0.45	0.11
500	8	0.974	9.921	0.10	0.01
2000	1	3.432	3.505	0.98	0.98

N	Hilos	T. secuencial (ms)	T. paralelo (ms)	Speedup	Eficiencia
2000	2	3.432	2.994	1.15	0.57
2000	4	3.432	3.011	1.14	0.29
2000	8	3.432	9.958	0.34	0.04
8000	1	14.531	14.648	0.99	0.99
8000	2	14.531	9.025	1.61	0.81
8000	4	14.531	6.077	2.39	0.60
8000	8	14.531	17.006	0.85	0.11
20000	1	36.819	37.195	0.99	0.99
20000	2	36.819	23.684	1.55	0.78
20000	4	36.819	12.367	2.98	0.74
20000	8	36.819	20.182	1.82	0.23
50000	1	93.322	96.725	0.96	0.96
50000	2	93.322	51.770	1.80	0.90
50000	4	93.322	28.305	3.30	0.82
50000	8	93.322	37.414	2.49	0.31

Datos crudos completos (250 filas, sin promediar): `results/benchmark_results.csv`. Resumen promediado: `results/benchmark_summary.csv`. Script usado: `scripts/run_benchmarks.sh`.

Verificación de corrección (semilla fija)

Con `-seed 12345`, `N=8000`, 300 frames:

Listing 4: Verificación: mismo resultado de simulación en todos los binarios/hilos

```

1 seq (1 hilo, serial): 26144 particulas recicladas
2 par --threads 1: 26144 particulas recicladas
3 par --threads 4: 26144 particulas recicladas
4 par --threads 8: 26144 particulas recicladas

```

Análisis

Ver el análisis completo, con la explicación de por qué `N` chico perjudica el paralelo y por qué 8 hilos rinde peor que 4, en `docs/anexo3_bitacora_pruebas.md` y en la sección 3.6 de este informe.

Referencias bibliográficas

1. Amdahl, G. M. “Validity of the single processor approach to achieving large scale computing capabilities”. *AFIPS Conference Proceedings*, vol. 30, 1967, pp. 483–485.
2. Foster, I. *Designing and Building Parallel Programs*. Addison-Wesley, 1995. <https://www.mcs.anl.gov/~itf/dbpp/>.
3. Grama, A., Gupta, A., Karypis, G., & Kumar, V. *Introduction to Parallel Computing* (2.^a ed.). Addison-Wesley, 2003.
4. OpenMP Architecture Review Board. *OpenMP Application Programming Interface, Version 5.2*. 2021. <https://www.openmp.org>.